

project

فكرة المشروع

- ساعة دائرية تحتوي على:
 - 3 عقارب متحركة (أحمر، أزرق، أخضر)
 - 12 نقطة بيضاء للساعات
 - إطار دائري قابل للتغيير
- التفاعل:
 - تغيير لون الإطار
 - تقريب وتباعد الإطار
 - التحكم بسرعة الوقت

-خطوات بناء المشروع

1-تهيئة البيئة (GLFW, GLEW)

2-إنشاء VBO, VAO, EBO

3-كتابة Vertex Shader

4-كتابة Fragment Shader

5-برمجة التفاعل مع الكيبورد

6-حلقة الرسم الرئيسية

المشاكل التي واجهتني

العقارب لا تتحرك .

الحل:

استخدام uniform uTime

إرسال Uniforms

// الحصول على موقع المتغير في الشيدر

```
GLint timeLoc = glGetUniformLocation(shaderProgram,  
"uTime");
```

// حساب الوقت (الوقت الحالي × السرعة)

```
float time = glfwGetTime() * speed;
```

// إرسال القيمة إلى الشيدر

```
glUniform1f(timeLoc, time);
```

Uniform استقبال واستخدام في Fragment Shader

```
// C++ استقبال الوقت من
uniform float uTime;

void main() {
    // استخدام الوقت لحساب زوايا العقارب
    float seconds = uTime;           // عدد الثواني
    float minutes = uTime / 60.0;     // عدد الدقائق
    float hours = uTime / 3600.0;     // عدد الساعات

    // (دورة كاملة =  $2\pi$ ) تحويل إلى زوايا
    float secondsAngle = seconds * 2.0 * 3.14159; // زاوية
    عقرب الثواني
    float minutesAngle = minutes * 2.0 * 3.14159; // زاوية عقرب
    الدقائق
    float hoursAngle = hours * 2.0 * 3.14159;    // زاوية عقرب
    الساعات

    // ... باقي الكود لرسم العقارب ...
}
```

// حساب المسافة من مركز الشاشة (0,0)

// تستخدم لتحديد الأشكال الدائرية
float dist = sqrt(x*x + y*y);

// (المسافة من المركز لمنتصف الإطار) نصف قطر الإطار
float frameRadius = 0.75;

// (قيمة صغيرة جداً لإطار رفيع) سمك الإطار
float frameWidth = 0.02;

// التحقق إذا كان البكسل ضمن منطقة الإطار الدائري
// الحافة الداخلية : $\text{dist} > \text{frameRadius} - \text{frameWidth}$
// الحافة الخارجية : $\text{dist} < \text{frameRadius} + \text{frameWidth}$
if (dist > frameRadius - frameWidth && dist < frameRadius + frameWidth) {

 // على اللون Z-Axis تطبيق تأثير

 // [0,1] إلى [-1,1] من المجال Z نحول

 // Z = 1.0 → (قريب) لون أفتح

 // Z = -1.0 → (بعيد) لون أغمق

 float zEffect = (uFrameZ + 1.0) / 2.0;

// حساب شفافية الإطار

// alpha = 1.0 : معتم

// alpha = 0.0 : شفاف تماماً

 float alpha = 1.0;

 if (uFrameZ < -0.3f) {

 alpha = 1.0 - (abs(uFrameZ) - 0.3f) * 1.5f;

 if (alpha < 0.0) alpha = 0.0; }

 // تلوين الإطار:

- // uFrameColor : (من C++ اللون الأساسي)

 // - zEffect : تأثير القرب والبعد

- // alpha : (للاختفاء التدريجي)

 FragColor = vec4(uFrameColor * zEffect, alpha)

 // ننهي البرنامج لهذا البكسل

 // حتى لا نكتب فوق لون الإطار

 return;

}